Université de Mons
Faculté des sciences
Département d'Informatique
Service d'Intelligence Artificielle

Deep Learning : Adaptation de domaine

Rapport de BIR

Chef de service :
Stéphane Dupont
Encadrant :
Yorick Estievenart

Auteur :
Yaël Wawrzyniak

Date : 29/09/2026





Année académique 2025-2026

Table des matières

1 Introduction **2**

2 Théorie **2**
2.1 Formulation du problème 2
2.2 Fonctionnement des modèles 3
2.3 Modèles 5
2.3.1 DANN 5
2.3.2 CIDA et PCIDA 5
2.4 $\mathcal{H}$-divergence 5
2.5 Hyperparamètre λ 6

3 Expériences **7**
3.1 *Intertwining Moons* 7
3.2 Autres expériences 10

4 Conclusion **12**

5 Références **12**

1 Introduction

En *Machine Learning*, on entraîne et teste un modèle sur deux *datasets* différents en supposant que la distribution des données d'entraînement est identique à celle des données de test. Cependant en réalité ces deux distributions sont différentes, on peut donc faire de l'*adaptation de domaine* des données d'entraînement vers les données de test.

L'adaptation de domaine est un problème d'apprentissage automatique dans lequel on retrouve deux domaines de données suivant des distributions différentes mais supposées similaires : un domaine source ('*source*') dans lequel les données sont labélisées et un domaine cible ('*target*') dans lequel elles ne le sont pas. Dans l'exemple précédent, les données d'entraînement constituent le domaine *source* tandis que les données de test constituent le domaine *target*. On peut également appliquer ce concept à un prédicteur en répartissant ses données en un domaine *source* et un domaine *target*. Par exemple : l'espèce d'un animal, la couleur de fond d'une image, l'âge d'un patient, etc.

Pour un modèle d'adaptation de domaine, il s'agit donc d'un apprentissage non supervisé sur le domaine *target* à l'aide de la *source*. Les modèles présentés sont entraînés de manière 'adverse' (*adversarial training*) : ils contiennent deux composantes antagonistes qui se renforcent l'une contre l'autre. Pour ce faire, le modèle doit extraire des caractéristiques les plus indépendantes possible des domaines pour pouvoir se rapprocher de la véritable distribution sous-jacente. Nous avons exploré plusieurs modèles conçus pour ce problème : DANN (Ganin et al., 2016) puis CIDA et PCIDA (Wang et al., 2020). Ce rapport présente la théorie liée à l'adaptation de domaine et les spécificités de chaque modèle ainsi que leurs performances sur des expériences inspirées de leurs articles d'origine.

2 Théorie

Nous avons choisi de reprendre le style de notation de Ganin et al., cette section s'inspire donc de leur article.

2.1 Formulation du problème

Considérons un problème de classification où X est l'espace d'entrée et $Y = \{0, \ldots, L-1\}$ est l'ensemble des labels possibles, avec L le nombre de

classes. On a deux distributions similaires sur $X \times Y$: le domaine *source* $\mathcal{D}_S$ et le domaine *target* $\mathcal{D}_T$ dont viennent respectivement les données *i.i.d. source* S (labélisées) et *target* T (non labélisées). Formellement, ces composantes sont définies par :

$$S = \{(\mathbf{x}_i, y_i, d_i)\}_{i=1}^{n} \sim (D_S)^n$$
$$T = \{\mathbf{x}_i, d_i\}_{i=n+1}^{N} \sim (D_T^X)^{n'}$$

avec $N = n + n'$ le nombre total de données et d_i le domaine. Ce dernier peut être soit discret (*e.g. source* : 0 et *target* : 1) soit continu (cf. section 2.3.2).

L'adaptation de domaine vise alors à estimer la distribution du domaine *target* le plus précisément possible à partir des données des domaines *source* et *target*.

2.2 Fonctionnement des modèles

L'architecture des modèles d'adaptation de domaine présentés (cf. figure 1) se compose d'un *feature extractor* $G_f(\cdot;\theta_f)$ prenant en entrée des données de S et T, d'un *label predictor* $G_y(\cdot;\theta_y)$ et d'un *domain classifier* $G_d(\cdot;\theta_d)$, aussi appelé discriminant, où θ_f, θ_y, θ_d sont les paramètres respectifs de G_f, G_y, G_d, avec G_y et G_d prenant en entrée la sortie de G_f. Le but du modèle, lors de l'entraînement, est de rendre le plus performant possible G_y tout en mettant en défaut G_d (G_y et G_d sont alors 'adversaires').

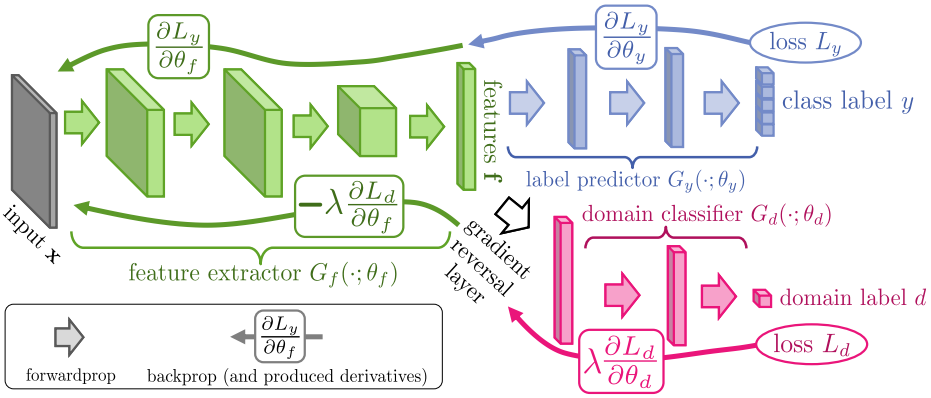


FIGURE 1 : Architecture générale des modèles présentés (Ganin et al., 2016).

Pour donner l'intuition derrière ce mécanisme, imaginons le cas suivant : notre modèle cherche à classer des images de nombres (style MNIST), le domaine *source* ayant des images à fond bleu tandis que le domaine *target*

aurait des images à fond jaune. Pour rappel, les données provenant du domaine *target* sont non labélisées donc G_y ne peut être évalué que sur la *source* S. Si G_y devient un excellent prédicteur de classe en s'entraînant uniquement sur les données de la *source* et que le discriminant G_d n'a aucun mal à différencier les domaines, alors G_f utilise comme caractéristique les différences liées au domaine (ici, le fond coloré). Résultat, notre prédicteur G_y sera très peu efficace face à des données du domaine *target* en situation réelle car il recevra de G_f des sorties dont il n'a pas l'habitude. À l'inverse, si G_d est perdu mais que G_y l'est aussi, on aura également un modèle faible.

Définissons les fonctions de perte pour les prédictions des classes et de domaine comme étant :

$$\mathcal{L}_y^i(\theta_f, \theta_y) = \mathcal{L}_y(G_y(G_f(\mathbf{x}_i; \theta_f); \theta_y), y_i),$$
$$\mathcal{L}_d^i(\theta_f, \theta_d) = \mathcal{L}_d(G_d(G_f(\mathbf{x}_i; \theta_f); \theta_d), d_i).$$

Pour rappel, les fonctions de perte mesurent l'écart entre la prédiction d'un modèle et la valeur attendue : plus cet écart est grand, plus le modèle se trompe. Elles permettent de faire converger les paramètres vers leurs valeurs optimales grâce à la *backpropagation* (gradients sur les pertes, symbolisé par les flèches arrières sur la figure 1).

Entraîner un modèle d'adaptation de domaine revient alors à optimiser :

$$E(\theta_f, \theta_y, \theta_d) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_y^i(\theta_f, \theta_y)}_{\substack{\text{Perte de } G_y \\ \text{sur la } \textit{source}}} - \lambda \Bigg(\underbrace{\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_d^i(\theta_f, \theta_d)}_{\substack{\text{Perte de } G_d \\ \text{sur la } \textit{source}}} + \underbrace{\frac{1}{n'}\sum_{i=n+1}^{N} \mathcal{L}_d^i(\theta_f, \theta_d)}_{\substack{\text{Perte de } G_d \\ \text{sur la } \textit{target}}} \Bigg).$$

où $\lambda \in \mathbb{R}^+$ est un hyperparamètre servant à gérer le compromis entre la performance de G_y et l'inefficacité de G_d. Si $\lambda = 0$, G_d n'a aucun impact sur l'entraînement du modèle (entraînement classique seulement sur G_y) et si λ est trop haut, il rendra l'amélioration de G_y sans importance.

Notez que dans la figure 1, la composante de la perte du domaine est multipliée par ce même λ et qu'elle subit une inversion de signe à la *Gradient Reversal Layer* (GRL) pour encourager le modèle à faire échouer le discriminant G_d. Cette couche n'a aucun paramètre associé et sert simplement à multiplier par -1 les gradients sur la perte du domaine.

2.3 Modèles

2.3.1 DANN

DANN (Domain Adversarial Neural Network, Ganin et al. 2016) est un modèle qui considère deux domaines discrets S, T et dont le discriminant prédit directement l'un ou l'autre domaine. Ses fonctions de perte (pour G_y et G_d) sont des *cross entropy loss* standards. C'est le modèle le plus simple que nous avons exploré, très proche de l'architecture décrite en figure 1.

2.3.2 CIDA et PCIDA

CIDA (Continuously Indexed Domain Adaptation, Wang et al. 2020) a un fonctionnement similaire à DANN mais présente une différence notable : ce modèle considère une ou plusieurs paires de domaines continus à la place d'une seule paire discrète. Le discriminant η de CIDA retourne donc des valeurs continues qu'on peut différencier avec un seuil de décision α tel que :

$$\mathbf{x} \sim \begin{cases} \mathcal{D}_S^X & \text{si } \eta(\mathbf{x}) \leq \alpha, \\ \mathcal{D}_T^X & \text{sinon.} \end{cases}$$

De plus, sa fonction de perte de domaine $\mathcal{L}_d$ est une perte *MSE* (*Mean Squared Error*), définie par :

$$\mathcal{L}_d(\eta(\mathbf{x}), d) = (\eta(\mathbf{x}) - d)^2.$$

PCIDA (Probabilistic CIDA, Wang et al. 2020) possède un discriminant η différent de CIDA qui prédit la moyenne η_μ et la variance η_{σ^2} de la distribution gaussienne $\mathbb{P}(d \mid \mathbf{x})$. Pour entraîner ce modèle, la fonction de perte de domaine est la *Negative Log Likelihood Loss* (*NLLLoss*), définie par :

$$\mathcal{L}_d(\eta(\mathbf{x}), d) = \frac{(\eta_\mu(\mathbf{x}) - d)^2}{2\eta_{\sigma^2}(\mathbf{x})} + \frac{1}{2} \log \eta_{\sigma^2}(\mathbf{x})$$

où $\eta(\mathbf{x}) = \big(\eta_\mu(\mathbf{x}), \eta_{\sigma^2}(\mathbf{x})\big)$.

2.4 $\mathcal{H}$-divergence

Considérons la classe d'hypothèses $\mathcal{H}$ suivante :

$$\mathcal{H} = \Big\{ \eta : X \to [0,1] : \mathbf{x} \to G_d(G_f(\mathbf{x}; \theta_f), \theta_d) \Big\}$$

comme étant l'ensemble des *domain classifiers* η possibles pour un modèle. La $\mathcal{H}$-divergence (Kifer et al., 2004) est une métrique permettant de quantifier la performance de η, définie par :

$$d_{\mathcal{H}}(\mathcal{D}_S^X, \mathcal{D}_T^X) = 2 \sup_{\eta \in \mathcal{H}} \left| \mathbb{P}_{\mathbf{x} \sim \mathcal{D}_S^X}[\eta(\mathbf{x}) = 1] - \mathbb{P}_{\mathbf{x} \sim \mathcal{D}_T^X}[\eta(\mathbf{x}) = 1] \right|.$$

Cette version est incalculable en pratique mais il existe une version empirique :

$$\hat{d}_{\mathcal{H}}(S, T) = 2 \left(1 - \min_{\eta \in \mathcal{H}} \left[\frac{1}{n} \sum_{i=1}^{n} I[\eta(\mathbf{x}_i) = 0] + \frac{1}{n'} \sum_{i=n+1}^{N} I[\eta(\mathbf{x}_i) = 1] \right] \right)$$

où $I[a]$ est la fonction indicatrice, égale à 1 si a est vraie, 0 sinon. On peut généraliser cette formule aux modèles à domaines continus :

$$\hat{d}_{\mathcal{H}}(S, T) = 2 \left(1 - \min_{\eta \in \mathcal{H}} \left[\frac{1}{n} \sum_{i=1}^{n} I[\eta(\mathbf{x}_i) \leq \alpha] + \frac{1}{n'} \sum_{i=n+1}^{N} I[\eta(\mathbf{x}_i) > \alpha] \right] \right)$$

où α est le seuil de décision entre *source* et *target*. Généralement, $\alpha = 0.5$.

Les deux moyennes dans le terme min représentent les performances de η sur les domaines *source* et *target*. Plus il est efficace, plus on se rapprochera de la valeur $2(1 - 0) = 2$, qui représente le pire scénario pour un modèle. À l'inverse, si η est inefficace, alors il ne fera pas mieux que le hasard : il prédira le bon domaine une fois sur deux. On tendra alors vers la valeur $2(1 - (0.5 + 0.5)) = 0$ qui constitue le meilleur cas pour un modèle.

Nous avons utilisé la $\mathcal{H}$-divergence empirique comme repère pour améliorer les discriminants de nos modèles et les comparer entre eux.

2.5 Hyperparamètre λ

Comme précisé auparavant, λ est un hyperparamètre qui quantifie l'importance de la perte du discriminant. On peut bien entendu fixer sa valeur une seule fois et la garder tout au long de l'entraînement, mais Ganin et al. ont aussi considéré un λ dynamique dont l'évolution est définie par :

$$\lambda = \frac{2}{1 + e^{-\gamma p}} - 1$$

où p représente le stade d'avancement de l'entraînement progressant linéairement de 0 à 1, et où γ est un hyperparamètre (que les auteurs ont toujours

fixé à 10) déterminant la vitesse de croissance de λ par rapport à p. L'intérêt d'un λ progressif réside dans le fait que le modèle peut d'abord se spécialiser sur la *source* et ensuite seulement adapter ses poids pour la *target*.

Cette formule est intéressante, cependant elle posait un problème : la valeur de λ est toujours comprise entre 0 et 1 alors qu'il peut être adéquat, selon le problème, d'avoir des valeurs supérieures à 1. Nous avons donc légèrement modifié la formule pour qu'elle soit plus flexible :

$$\lambda = \lambda_{\max}\left(\frac{2}{1+e^{-\gamma p}} - 1\right)$$

où $\lambda_{\max}$ est un hyperparamètre déterminant la valeur maximale que peut prendre λ (dans le cas précédent, $\lambda_{\max} = 1$).

3 Expériences

Pour comparer les modèles entre eux, nous avons réalisé des expériences en utilisant comme principal package PyTorch [1] afin de construire et entraîner nos modèles. Ces derniers ayant deux modes différents concernant λ (fixé ou progressif), nous avons utilisé Optuna [2] pour sélectionner le ou les hyperparamètres nécessaires à ces modes. Le code sera bientôt disponible sur le dépôt suivant : https://github.com/Yayel4289/BIR.

Nous avons commencé par construire un modèle de base (*baseline*) ne pouvant pas utiliser les données du domaine *target*. DANN, CIDA et PCIDA ont le même prédicteur G_y que la *baseline*, avec en plus leurs discriminants G_d et leurs spécificités. Notre but, dans une expérience, est que tous les modèles d'adaptation de domaine soient plus performants que la *baseline*, puisqu'elle n'implémente pas la théorie présentée auparavant.

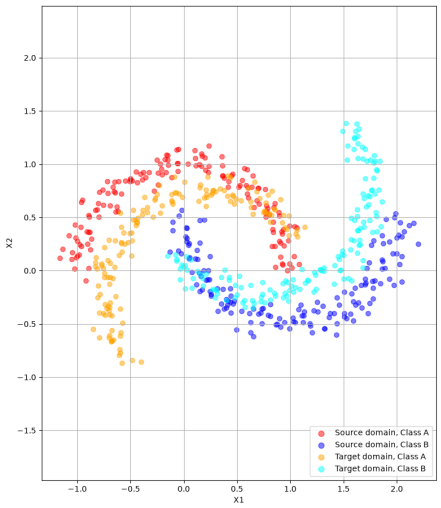
3.1 *Intertwining Moons*

Nous nous sommes inspirés du *toy problem* "*Intertwining Moons*" que Ganin et al. ont réalisé pour DANN. C'est un problème de classification sur domaines discrets ou continus dont vous pouvez voir des illustrations sur les figures 2a à 2d. Les figures 2a et 2c montrent le problème sous sa forme discrète (resp. continue) tandis que les figures 2b et 2d révèlent le problème

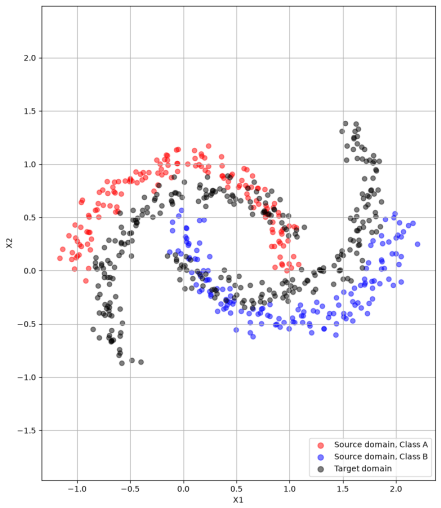
1. https://pytorch.org/
2. https://optuna.org/

vu par un modèle discret (resp. continu) car pour rappel, les données du domaine *target* sont non labélisées.

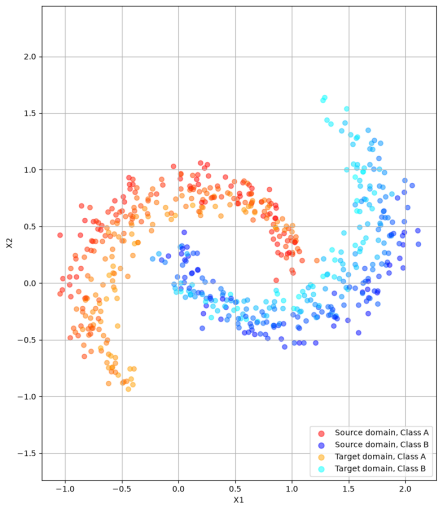
Le domaine réside ici dans l'angle θ d'inclinaison des paires de lunes (classes A et B). Dans le cas discret, le domaine *source* n'est pas incliné ($\theta = 0$ rad) tandis que les points du domaine *target* subissent une rotation de $\theta = \frac{\pi}{5}$rad par rapport au centre du plan. Dans le cas continu, les points subissent une rotation graduelle ($\theta \in [0, \frac{\pi}{5}]$). Si la rotation subie par un point est supérieure à $\frac{\pi}{10}$rad, il appartient au domaine *target*, sinon, au domaine *source*.



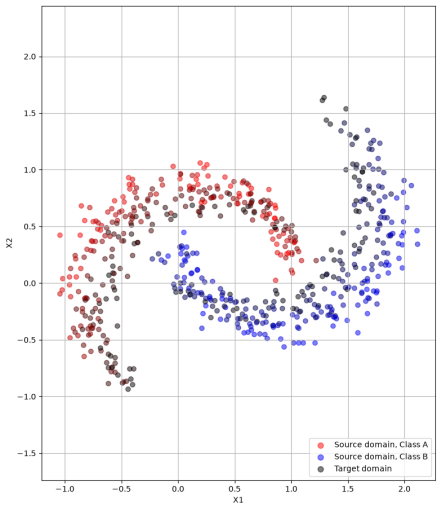
(a) Domaines discrets.



(b) Figure 2a, vu par un modèle.



(c) Domaines continus.



(d) Figure 2c, vu par un modèle.

FIGURE 2 : *Intertwining Moons* avec des domaines discrets (en haut) et continus (en bas), à gauche telles que définies, à droite telles que vues par un modèle.

L'objectif consiste à pouvoir distinguer les deux classes A et B, notamment sur le domaine *target* (entraîner un classificateur efficace sur la *source* n'est généralement pas un problème). Pour mesurer les performances des mo-

dèles, nous avons utilisé la $\mathcal{H}$-divergence (cf. section 2.4) ainsi que le *F1 Score* (version *macro*) sur le domaine *source* et le domaine *target*. Nous avons comparé tous les modèles ainsi que leurs modes sur ces métriques, chaque modèle ayant son environnement adapté (discret ou continu, cf. figure 2). Voici les résultats obtenus :

Modèles \ Métriques		$\lvert\mathcal{H}$-divergence$\rvert$	Source F1 Score	Target F1 Score
Baseline		N/A	0.999 ± 0.001	0.717 ± 0.040
DANN	λ fixe	0.045 ± 0.069	0.991 ± 0.006	0.768 ± 0.138
	λ progressif	0.078 ± 0.328	0.998 ± 0.002	0.807 ± 0.123
CIDA	λ fixe	0.256 ± 0.095	0.999 ± 0.001	$\mathbf{0.940 \pm 0.042}$
	λ progressif	0.028 ± 0.097	0.999 ± 0.001	$\mathbf{0.934 \pm 0.033}$
PCIDA	λ fixe	0.000 ± 0.000	0.999 ± 0.001	$\mathbf{0.953 \pm 0.042}$
	λ progressif	0.000 ± 0.000	0.992 ± 0.008	$\mathbf{0.922 \pm 0.062}$

TABLE 1 : Résultats obtenus (meilleures valeurs en gras).

Nos modèles dominants ont donc été CIDA et PCIDA puisqu'ils ont de loin les meilleures performances sur le domaine *target*. Nos résultats vont donc dans le sens de Wang et al., même s'il est possible que DANN soit plus performant sur d'autres problèmes.

3.2 Autres expériences

Avant de tester nos modèles sur les *Intertwining Moons*, nous les avons essayés sur d'autres configurations que vous pouvez voir sur les figures 3 et 4. Nous avons remplacé ces problèmes par les *Intertwining Moons* car la *baseline* pouvait apprendre une bonne frontière de décision sans utiliser les méthodes d'adaptation de domaine. Elle n'avait donc pas de grands écarts de performance avec les autres modèles (elle pouvait même en surpasser), nous avons alors conclu que nous devions changer l'environnement des modèles et avons choisi les *Intertwining Moons*

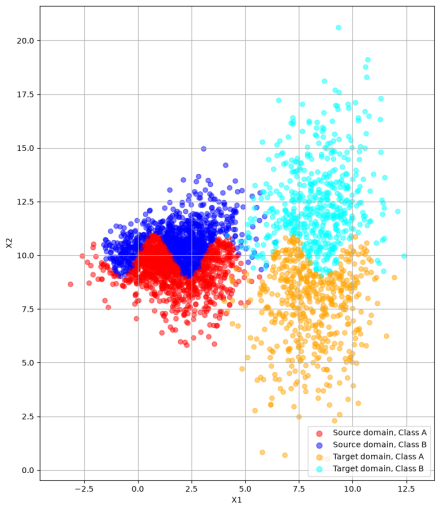


FIGURE 3 : Nuages de points gaussiens, domaines séparés.

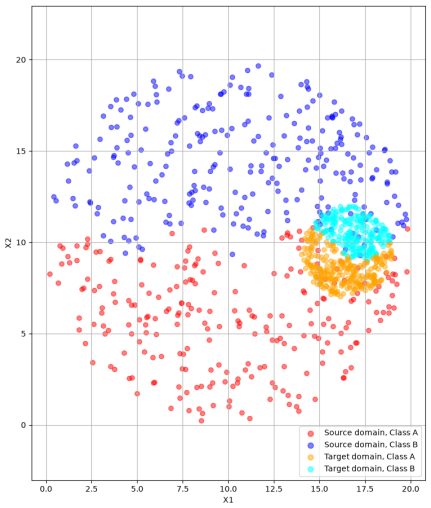


FIGURE 4 : Nuages de points circulaires, domaines enchevêtrés.

Nous voulions également entraîner nos modèles sur un jeu de données médicales. Nous avions soumis une demande à la NSRR (National Sleep Research Resource [3]) pour obtenir l'accès aux *datasets* qu'utilisent Wang et al. dans les expériences sur CIDA, mais malheureusement la demande a d'abord été refusée, puis acceptée trop tard pour nous. Nous avons donc pris un autre jeu de données, en accès libre cette fois : Skin Cancer MNIST : HAM10000 [4], dans lequel on peut trouver des images (résolution complète ou 28×28 pixels, cf. figures 5 et 6) de lésions cutanées et leur maladie associée. L'objectif pour nos modèles était de distinguer les mélanomes (cancéreux, cf. figures 5 et 6) des nævi congénitaux mélanocytaires (bénins) à partir des images de 28×28 pixels car une plus grande résolution aurait été trop lente. Les données étaient séparées en deux domaines *source* et *target* selon l'âge des patients : si leur âge était inférieur à 30 ans, ils faisaient partie du domaine *target*, sinon du domaine *source*. Hélas, les résultats ont été peu concluants, nous pensons que c'est à cause de la résolution trop basse des images en entrée. Nous aurions pu réessayer avec une plus grande résolution mais le stage arrivait à son terme après ce dernier essai.

3. https://sleepdata.org/
4. https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000

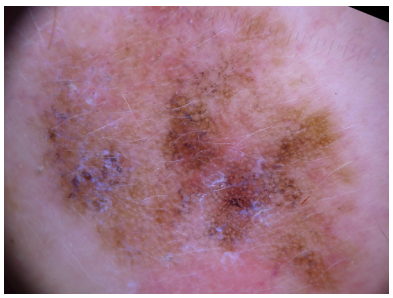


FIGURE 5 : Image de mélanome précoce du Skin Cancer MNIST.

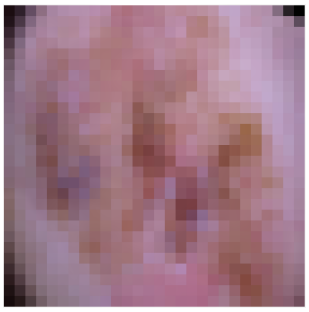


FIGURE 6 : Figure 5 en 28×28 pixels du Skin Cancer MNIST.

4 Conclusion

Pour conclure, les modèles à entraînement adverse (*adversarial training*) présentent un compromis entre l'efficacité du *label predictor* et l'incapacité du discriminant, ce dernier pouvant être mesuré grâce à la $\mathcal{H}$-divergence. Ce compromis est géré par l'hyperparamètre λ qui peut être fixe ou évoluer progressivement pendant l'entraînement. Nos expériences semblent montrer que ces deux modes pour λ ont des performances similaires, et que CIDA et PCIDA sont plus efficaces que DANN quand on peut représenter les domaines de manière continue. De manière générale, tous les modèles implémentant l'adaptation de domaine se sont révélés plus performants que la *baseline* sur la *target*, indiquant que ce concept est bel et bien efficace.

Enfin, je souhaite remercier Infortech [5] et Stéphane Dupont pour m'avoir permis d'effectuer ce stage au sein de l'UMONS, ainsi que mon encadrant Yorick Estievenart qui m'a guidé tout au long de ce travail.

5 Références

Yaroslav Ganin, Evgeniya Ustinova, Hana Ajakan, Pascal Germain, Hugo Larochelle, François Laviolette, Mario Marchand, and Victor Lempitsky. Domain-adversarial training of neural networks, 2016. URL https://arxiv.org/abs/1505.07818.

Daniel Kifer, Shai Ben-David, and Johannes Gehrke. Detecting change in data streams. In *Proceedings of the Thirtieth International Conference on*

5. https://web.umons.ac.be/infortech/

Very Large Data Bases, volume 30, pages 180–191. VLDB Endowment, 2004.

Hao Wang, Hao He, and Dina Katabi. Continuously indexed domain adaptation, 2020. URL https://arxiv.org/abs/2007.01807.